

NoteNest Portal

Complete Code Documentation

Comprehensive Technical Catalog of all HTML5 Semantic Tags, CSS3 Design System Rules & Properties, and Vanilla JavaScript Functions used in the Project.

PROJECT NAME

NoteNest - Students Notes Sharing Portal

TECHNOLOGY STACK

Pure HTML5, CSS3, Vanilla JS (ES6+)

ARCHITECTURE

Multi-Page + Multi-Script + LocalStorage

TARGET FRAMEWORKS

Zero External Libraries (100% Native)

TOTAL HTML TAGS USED

48 Distinct HTML5 & SVG Elements

TOTAL CSS PROPERTIES

119 CSS3 Properties & 35 Custom Tokens

TOTAL JAVASCRIPT FUNCTIONS

42 Modular & Documented Functions

RESPONSIVE SUPPORT

Mobile (320px) to 4K Ultrawide (1440px+)



Executive Summary & Architecture

NoteNest is built from the ground up strictly according to pure web standards. It requires zero backend servers, zero build steps, and zero third-party UI toolkits. Every component, transition, responsive drawer, modal dialog, and data synchronization is handcrafted using native web APIs.



6 HTML Pages



7 Modular JS Files



1 Comprehensive CSS File (1,900+ Lines)



5 LocalStorage Entities



100% Mobile Responsive



File Manifest Overview

FILE NAME	TYPE	PRIMARY RESPONSIBILITY & ROLE
index.html / home.js	Landing Page	Hero section with search, animated category grid, dynamic trending notes counter, and feature walkthrough.
explore.html / explore.js	Explorer View	Multi-criteria filtering engine (Department, Semester, Subject, Note Type, Rating), search, sorting, and active filter chips.
upload.html / upload.js	Upload Portal	Drag-and-drop file upload dropzone simulation, real-time client-side validation, error messages, and immediate data publishing.
dashboard.html / dashboard.js	Student Dashboard	Real-time greeting, dynamic analytics metric recalculator, uploaded notes table with deletion, saved bookmarks, and recently viewed history.
profile.html / profile.js	Student Profile	Profile card, dynamic badge calculations, stats overview, "Edit Profile" modal dialog, and one-click demo data reset.
about.html / about.js	About Platform	Platform vision, mission pillars, architecture principles, and interactive collapsible FAQ accordion.
common.js	Shared Core Engine	Master data state, LocalStorage manager, Theme engine (Light/Dark), Auth simulation, Modals, Toast notifications, and PDF preview simulator.
style.css	Global Design System	CSS Custom Properties (:root variables), responsive fluid typography, glassmorphism, responsive grid & flexbox, off-canvas drawers, and animations.

Section 1: Complete Catalog of HTML Tags Used

The project uses **48 semantic HTML5 and SVG vector tags**. Below is the complete catalog with attributes, purpose, and usage in NoteNest.

TAG	CATEGORY	ATTRIBUTES USED	ROLE & USAGE IN NOTENEST
<code><html></code>	Root Element	<code>lang="en"</code>	Defines root document and declares English language for accessibility.
<code><head></code>	Document Meta	None	Contains document metadata, title, viewport configuration, and stylesheet links.
<code><meta></code>	Document Meta	<code>charset</code> <code>name</code> <code>content</code>	Configures UTF-8 character encoding, responsive viewport width, and SEO descriptions.
<code><title></code>	Document Meta	None	Sets the dynamic browser tab title for each individual page.
<code><link></code>	Resource Link	<code>rel</code> <code>href</code>	Links the shared <code>style.css</code> and Google Fonts ('Plus Jakarta Sans' and 'Fira Code').
<code><body></code>	Document Body	<code>class</code>	Main view container; receives the <code>dark-theme</code> class dynamically via JavaScript.
<code><header></code>	Semantic Layout	<code>class</code>	Sticky top navigation bar containing logo, nav links, global search trigger, theme toggle, and auth button.
<code><nav></code>	Semantic Layout	<code>class</code> <code>aria-label</code>	Primary navigation container and mobile off-canvas drawer navigation.
<code><main></code>	Semantic Layout	<code>class</code> <code>id</code>	Wraps the unique primary content for each page for accessibility and screen readers.
<code><section></code>	Semantic Layout	<code>class</code> <code>id</code>	Divides pages into logical chunks: Hero, Categories, Trending, Filters, FAQs, and Dashboard Stats.
<code><aside></code>	Semantic Layout	<code>class</code> <code>id</code>	Houses the filter sidebar on <code>explore.html</code> and off-canvas mobile filter drawer.
<code><footer></code>	Semantic Layout	<code>class</code>	Site-wide footer displaying brand tagline, quick links, category links, and copyright info.
<code><h1></code>	Typography	<code>class</code>	Page titles (e.g., "Your Notes. Your Community. Your Success." on Home, "Explore Study Notes").

TAG	CATEGORY	ATTRIBUTES USED	ROLE & USAGE IN NOTENEST
<code><h2></code>	Typography	<code>class</code>	Major section headings (e.g., "Popular Categories", "My Uploaded Notes", "Platform FAQs").
<code><h3></code>	Typography	<code>class</code>	Card titles (Note titles, Feature card headings, Metric card labels, Modal titles).
<code><h4></code>	Typography	<code>class</code>	Sidebar filter group titles (e.g., "Department", "Semester", "Minimum Rating").
<code><p></code>	Typography	<code>class</code>	Paragraph descriptions, note descriptions, subtitle text, and form guidance copy.
<code></code>	Inline Text	<code>class</code> <code>id</code>	Badges (PDF, Semester, Rating), counter values, uploader initials, and status indicators.
<code></code>	Inline Text	None	Emphasizes important terms, labels, and statistics values.
<code><small></code>	Inline Text	<code>class</code>	Field validation error messages (e.g., <code>.field-error</code>) and helper notes.
<code><kbd></code>	Inline Text	None	Displays keyboard shortcuts in the search bar (e.g., <code>Ctrl + K</code>).
<code><a></code>	Hyperlink	<code>href</code> <code>class</code> <code>target</code>	Navigation links, category cards with URL query parameters (e.g., <code>explore.html?dept=...</code>).
<code><button></code>	Interactive	<code>type</code> <code>class</code> <code>id</code> <code>onclick</code>	Triggers actions: bookmarking, downloads, modal open/close, tabs, theme toggles, and filters.
<code><form></code>	Form Container	<code>id</code> <code>class</code> <code>novalidate</code>	Upload notes form, login/signup form, and profile editor modal form.
<code><input></code>	Form Input	<code>type</code> <code>placeholder</code> <code>required</code>	Text search, file upload input (<code>type="file"</code>), radio buttons for rating, email & password.
<code><select></code>	Form Select	<code>id</code> <code>class</code> <code>required</code>	Dropdown pickers for Department, Semester, Note Type, Subject, and Sort Order.
<code><option></code>	Form Option	<code>value</code> <code>selected</code>	Defines selectable choices within <code><select></code> elements.
<code><textarea></code>	Form Textarea	<code>rows</code> <code>placeholder</code>	Multi-line inputs for note descriptions and user biography on profile.
<code><label></code>	Form Label	<code>for</code> <code>class</code>	Accessible form field labels and custom styled radio/checkbox containers.

TAG	CATEGORY	ATTRIBUTES USED	ROLE & USAGE IN NOTENEST
<code><table></code>	Data Table	<code>class</code>	Structures the "My Uploaded Notes" data grid in <code>dashboard.html</code> .
<code><thead></code>	Data Table	None	Groups table header rows for columns: Note Title, Subject, Date, Downloads, Status, Actions.
<code><tbody></code>	Data Table	<code>id</code>	Dynamic target filled with note rows via <code>dashboard.js</code> .
<code><tr></code>	Data Table	None	Defines rows inside table headers and dynamic table bodies.
<code><th></code>	Data Table	None	Defines header cells with uppercase titles.
<code> / </code>	Lists	<code>class</code>	Navigation link lists, footer links, notification drop lists, and upload guidelines.
<code><hr></code>	Divider	<code>class</code>	Semantic horizontal divider lines separating sections and modal footers.
<code>
</code>	Line Break	None	Forces clean line breaks in hero banners and headers.
<code><div></code>	Generic Box	<code>class</code> <code>id</code>	The core container for flexbox rows, grids, cards, badges, dropzones, modals, and toasts.
<code><svg></code>	Vector Graphics	<code>viewBox</code> <code>fill</code> <code>stroke</code>	Scalable crisp UI icons (Search, Moon, Sun, Bell, Download, Star, Filter, Close, Bookmark).
<code><path></code>	SVG Path	<code>d</code>	Defines complex vector geometry for all custom UI icons.
<code><circle></code>	SVG Shape	<code>cx</code> <code>cy</code> <code>r</code>	Renders circular elements in notification bell badges and illustration icons.
<code><rect></code>	SVG Shape	<code>x</code> <code>y</code> <code>width</code> <code>rx</code>	Renders file preview icons and badge backgrounds in SVG.
<code><polygon> / <polyline></code>	SVG Shape	<code>points</code>	Renders 5–star rating icons, checkmarks, and arrow chevrons.
<code><line></code>	SVG Shape	<code>x1</code> <code>y1</code> <code>x2</code>	Renders cross 'X' close buttons and decorative grid lines.
<code><script></code>	Script Hook	<code>src</code> <code>defer</code>	Loads modular JavaScript logic (<code>common.js</code> and specific page scripts).

Section 2: Complete CSS3 Architecture & Styling Properties

The styling system is encapsulated entirely within `style.css`. It implements a modern **Student-Tech Design System** with CSS custom properties (variables), responsive layout primitives, fluid typography, smooth cubic-bezier transitions, and glassmorphism.

CSS Variables & Design Tokens (:root)

VARIABLE NAME	LIGHT MODE VALUE	PURPOSE & USAGE
--primary	#2563eb (Royal Blue)	Primary brand color for CTA buttons, active links, and primary accents.
--primary-hover	#1d4ed8 (Deep Blue)	Interactive hover state for buttons and clickable chips.
--primary-light	#eff6ff (Ice Blue)	Light background tint for active badges, chips, and table hover states.
--secondary	#7c3aed (Violet Purple)	Secondary accent used in category badges, gradients, and illustration accents.
--accent-orange	#f59e0b (Amber Orange)	High-visibility badge for popular notes, stars rating, and hot tags.
--accent-green	#10b981 (Emerald Green)	Success toast notifications, verified student badges, and download icons.
--bg-body	#f8fafc (Slate 50)	Main canvas background (switches to #0b0f19 in Dark Theme).
--bg-card	ffffff (Pure White)	Elevated surface color for note cards, modals, and dropdowns.
--text-main	#0f172a (Slate 900)	Primary high-contrast text for headers and strong elements.
--text-muted	#64748b (Slate 500)	Subtitles, meta labels, timestamps, and placeholder text.
--border-color	#e2e8f0 (Slate 200)	Subtle 1px border lines for cards, tables, inputs, and dividers.
--shadow-sm/md/lg	0 1px 2px ... 0 20px 25px	Elevation layers providing realistic depth without heavy shadows.

VARIABLE NAME	LIGHT MODE VALUE	PURPOSE & USAGE
--radius-md/lg/xl	8px / 12px / 20px	Rounded modern student-tech aesthetic for cards, inputs, and modals.
--transition	all 0.25s cubic-bezier(0.4, 0, 0.2, 1)	Silky smooth acceleration curves for transforms and color fades.

Complete Categorized List of CSS Properties Used

CATEGORY	PROPERTIES IMPLEMENTED IN STYLE.CSS	TECHNICAL APPLICATION IN NOTENEST
Layout & Flexbox	display, flex, flex-direction, flex-wrap, align-items, justify-content, gap, flex-grow, flex-shrink	Forms header bars, button groups, badge chips, modal footers, and aligned icon-label pairs.
CSS Grid	grid-template-columns, grid-template-rows, grid-gap, repeat(), minmax(), auto-fit, auto-fill	Responsive note card grids (1 to 3 columns auto-adapting) and category card matrices.
Positioning & Depth	position (sticky, fixed, absolute, relative), top, right, bottom, left, z-index, inset	Sticky navbar, fixed modal backdrop overlay (z-index: 1000), toast stacks, and badge pins.
Box Model & Sizing	width, height, min-width, max-width, min-height, max-height, margin, padding, box-sizing, overflow, overflow-y, overflow-x	Container constraints (max-width: 1280px), card padding, and scrollable modal bodies.
Typography & Fonts	font-family, font-size, font-weight, line-height, letter-spacing, text-align, text-transform, text-decoration, text-overflow, white-space, clamp()	Fluid heading sizes using clamp(1.75rem, 4vw, 2.75rem) and single-line ellipsis for long note titles.
Colors & Backgrounds	color, background, background-color, background-image, linear-gradient(), radial-gradient(), opacity	Modern gradient hero banners, badge colorations, and dark-theme contrast switches.
Borders & Radius	border, border-width, border-style, border-color, border-radius, border-top, border-bottom, outline, outline-offset	Rounded card aesthetics, focus rings for accessible tab navigation, and table dividers.
Shadows & Glassmorphism	box-shadow, text-shadow, backdrop-filter: blur(), -webkit-backdrop-filter	Glass-like sticky navigation bar with blur(12px) and floating card shadows.
Transforms & Motion	transform (translate, translateY, scale), transform-origin, transition, transition-property, transition-duration, transition-timing-function	Subtle translateY(-4px) hover lift effect on note cards and smooth modal pop-in animations.
Keyframe Animations	@keyframes, animation, animation-name, animation-duration, animation-timing-function, animation-fill-mode	modalFadeIn, toastSlideIn, pulseGlow, and shimmerLoading animations.

CATEGORY	PROPERTIES IMPLEMENTED IN STYLE.CSS	TECHNICAL APPLICATION IN NOTENEST
Interactive & Pseudo	:hover, :focus, :focus-visible, :active, :disabled, :checked, :nth-child(), ::placeholder, ::before, ::after, ::selection	Input focus glow states, radio button custom icons, pseudo-element gradient badges.
Responsive Breakpoints	@media (max-width: 1024px), @media (max-width: 768px), @media (max-width: 480px), @media print	Adaptive layouts converting sidebar into slide-in drawer and multi-column grids into single-column mobile views.

⚡ Section 3: Complete JavaScript Architecture & All Functions

The application employs a pure Vanilla JavaScript ES6+ architecture. All data synchronizes through a unified client-side state manager and persistent `localStorage` database.

📦 1. `common.js` (Shared Foundation & Core System)

Contains data models, initial seed notes, storage synchronization, theme management, modal controllers, and notification systems.

`initCommon()`

Scope: Self-Invoking DOMReady

Initializes the shared site-wide subsystem on every page load: attaches global event listeners, binds shortcuts (Ctrl+K), checks theme, and initializes auth UI.

Parameters: None | Returns: void

`initDataStore()`

Scope: Core Storage Controller

Checks `localStorage` for initial datasets (`notenest_notes`, `notenest_user`, `notenest_bookmarks`, `notenest_recent`). If empty, injects rich seed notes.

Parameters: None | Returns: void

`initTheme()`

Scope: Theme Controller

Reads user theme preference from `localStorage.getItem('notenest_theme')` or system preference. Toggles `dark-theme` class on `<body>`.

Parameters: None | Returns: void

`toggleTheme()`

Scope: Exported (`window.toggleTheme`)

Switches theme between Light and Dark mode, updates SVG icons (Sun/Moon), saves preference, and triggers notification.

Parameters: None | Returns: void

createNoteCardHTML(note)

Scope: Global Component Generator

Generates the complete responsive HTML template string for a Note Card, including subject badges, bookmark state, download counts, and preview triggers.

Parameters: note (Object) | Returns: String (HTML)

openNoteModal(noteId)

Scope: Exported (window.openNoteModal)

Finds note by ID, populates the multi-page preview modal with title, uploader info, tags, and paginated content, adds note to "Recently Viewed", and opens modal.

Parameters: noteId (Number) | Returns: void

toggleBookmark(noteId, event)

Scope: Exported (window.toggleBookmark)

Adds or removes note ID from NoteNest.bookmarks array, syncs to localStorage, fires toast, and notifies active page listeners via window.onBookmarksUpdate.

Parameters: noteId (Number), e (Event) | Returns: void

downloadNote(noteId, event)

Scope: Exported (window.downloadNote)

Simulates real file download: increments note download counter in state, triggers browser text file download via Blob URI, updates UI counters, and shows toast.

Parameters: noteId (Number), e (Event) | Returns: void

showToast(title, message, type)

Scope: Exported (window.showToast)

Dynamically mounts a self-dismissing toast notification card into the toast container with entrance and exit animations.

Parameters: title (String), message (String), type (String: 'success'|'info'|'warning'|'error') | Returns: void

openModal(modalEl) / closeModal(modalEl)

Scope: Modal Dialog Controllers

Handles opening/closing of modal backdrops with smooth transitions and locks body scroll.

Parameters: modalEl (HTMLElement) | Returns: void

setupNavbar() / setupDrawer()

Scope: Responsive Navigation Handlers

Controls mobile hamburger toggle button, off-canvas navigation slide-out drawer, and outside-click dismissal listeners.

Parameters: None | Returns: void

renderUserAuthSlot() / setupAuthModal()

Scope: Simulated Auth Controllers

Renders user login avatar dropdown in navbar or login button, handles login/signup form submissions, validates credentials, and syncs session state.

Parameters: None | Returns: void

2. home.js (Landing Page Logic)

`initHomePage()`

Scope: DOMContentLoaded Listener

Orchestrates home page rendering by invoking category counter calculator and trending notes generator.

Parameters: None | Returns: void

`renderCategories()`

Scope: Component Renderer

Iterates over department categories, computes real-time note count for each category, and renders clickable category cards linking to `explore.html?dept=...`

Parameters: None | Returns: void

`renderTrendingNotes()`

Scope: Component Renderer

Sorts all notes by downloads descending (`[...NoteNest.notes].sort((a,b) => b.downloads - a.downloads)`) and displays top 3 cards in trending section.

Parameters: None | Returns: void

3. explore.js (Notes Explorer & Filtering Engine)

`initExplorePage()`

Scope: DOMContentLoaded Listener

Parses URL search parameters (`dept`, `search`), populates subject dropdown options dynamically, binds input listeners, and triggers initial filter pipeline.

Parameters: None | Returns: void

`applyFiltersAndRender()`

Scope: Core Filtering Engine

Executes multi-criteria matching across: Search Query (title, subject, uploader, tags), Department, Semester, Note Type, Subject, and Minimum Rating. Sorts and renders resulting cards.

Parameters: None | Returns: void

`renderActiveFilterChips()`

Scope: UI State Visualizer

Constructs interactive removable filter chips for all currently active filter criteria above the results grid.

Parameters: None | Returns: void

resetAllFilters()

Scope: Filter Reset Handler

Clears all search inputs, resets dropdowns to "All", unchecks rating radios, and updates results grid.

Parameters: None | Returns: void

removeFilterChip(filterKey)

Scope: Exported (window.app.removeFilterChip)

Removes a single specific active filter when the user clicks its 'X' chip button and re-runs the filter pipeline.

Parameters: filterKey (String) | Returns: void



4. upload.js (Notes Publishing & Dropzone Simulation)

initUploadPage()

Scope: DOMContentLoaded Listener

Sets up drag-and-drop file dropzone listeners (dragenter, dragover, dragleave, drop), validates file input, and binds form submission.

Parameters: None | Returns: void

handleFileObject(file)

Scope: Dropzone File Processor

Validates file format (PDF, DOCX, PPTX, TXT, ZIP) and size (< 25MB), updates UI dropzone with file name and size badge.

Parameters: file (File) | Returns: Boolean

handleUploadSubmit(e)

Scope: Form Submission Handler

Validates all mandatory fields, generates new note object, prepends into `NoteNest.notes`, saves to `localStorage`, increments user uploads counter, and redirects to `explore.html`.

Parameters: e (Event) | Returns: void



5. dashboard.js (Student Dashboard & Live Metrics)

initDashboardPage()

Scope: DOMContentLoaded Listener

Computes time-of-day greeting (Morning / Afternoon / Evening), recalculates live analytics, populates tabs, and binds reactive update hooks.

Parameters: None | Returns: void

renderMetrics()

Scope: Analytics Calculator

Dynamically recalculates metric counter cards: Notes Uploaded, Total Downloads received, Saved Bookmarks count, and Profile Views.

Parameters: None | Returns: void

renderTabs()

Scope: Tab Content Renderer

Renders: 1. "My Uploaded Notes" interactive table, 2. "Saved Notes" grid, and 3. "Recently Viewed" historical notes stream.

Parameters: None | Returns: void

deleteMyNote(noteId)

Scope: Exported (window.app.deleteMyNote)

Deletes an uploaded note from state and localStorage, displays toast, and re-renders table and metric cards without page refresh.

Parameters: noteId (Number) | Returns: void



6. profile.js & ? 7. about.js

renderProfileDetails() (profile.js)

Scope: Profile Data Binder

Populates profile banner with avatar initials, student name, email, department, semester, college name, bio, badges, and upload statistics.

Parameters: None | Returns: void

setupProfileEditor() (profile.js)

Scope: Modal Profile Editor

Handles live profile editing in modal dialog: updates `NoteNest.user`, computes new avatar initials, syncs to `localStorage`, and notifies components.

Parameters: None | Returns: void

`initAboutPage()` (`about.js`)

Scope: FAQ Accordion Controller

Binds click events to FAQ accordion buttons to expand/collapse answer panels with smooth height transitions and rotates chevron icons.

Parameters: None | Returns: void



Section 4: College Viva–Voce Quick Reference Guide

Essential questions and model answers to help explain this project during technical evaluations, lab exams, and viva presentations.

Q1: How does NoteNest achieve data persistence without a backend or MySQL database?

Answer: NoteNest utilizes the HTML5 `localStorage` Web Storage API. JSON-serialized datasets (`notenest_notes`, `notenest_user`, `notenest_bookmarks`, `notenest_recent`) are stored locally in the user's browser client. Any note uploads, bookmark toggles, profile edits, or theme switches are serialized with `JSON.stringify()` and retrieved with `JSON.parse()` upon page load.

Q2: How does the application filter notes across multiple criteria simultaneously?

Answer: The filtering engine in `explore.js` uses JavaScript's functional array pipeline (`Array.prototype.filter()` and `Array.prototype.sort()`). It evaluates multiple predicates sequentially: search query regex/string matching against title, subject, uploader, and tags; strict equality checks for Department, Semester, Note Type, and Subject; and numerical comparison for Minimum Rating.

Q3: How is responsive layout achieved across mobile, tablet, and desktop without Bootstrap or Tailwind?

Answer: Responsiveness is built using pure CSS3 Flexbox, CSS Grid with dynamic repeat formulas (`grid-template-columns: repeat(auto-fill, minmax(320px, 1fr))`), and CSS Media Queries at 1024px, 768px, and 480px. On smaller screens, the desktop filter sidebar automatically transforms into an off-canvas slide-out drawer triggered by JavaScript.

Q4: How do different pages communicate and share updated state?

Answer: All pages include the shared `common.js` module before their specific script. `common.js` manages the global NoteNest object and exports reactive hooks (`window.onBookmarksUpdate`, `window.onNotesUpdate`, `window.onUserUpdate`). When an action occurs on any page or modal, these hooks trigger automatic component re-rendering.

Q5: How is simulated file download implemented in pure JavaScript?

Answer: When a user clicks "Download Note", `downloadNote()` creates a temporary text Blob in memory using `new Blob([content], { type: 'text/plain' })`. It then creates a virtual `<a>` element, assigns `URL.createObjectURL(blob)` to its href, sets the `download` attribute to the note's filename, and programmatically triggers `a.click()` to initiate the browser download.